

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবেরো

(Prof. Chester Rebeiro)

কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ

(Department of Computer Science and Engineering),

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাজ

(Indian Institute of Technology, Madras)

সপ্তাহ – 07

বক্তৃতা – 32

ডেডলক পরিচালনা

(Dealing with Deadlocks)

Deadlock পরিচালনা করার জন্য 3 টি উপায় রয়েছে। যথা deadlock থেকে detection and recovery (নির্ণয় ও নিরাময়), deadlock avoidance (এড়ানো) এবং deadlock prevention (প্রতিরোধ)। এই ভিডিওতে, আমরা deadlock avoidance এবং prevention এর উপর নজর রাখব। Avoidance দ্বারা আমরা বলতে চাচ্ছি যে, সিস্টেম এমন একটি state এ যেতে পারবে না যা সম্ভাব্য deadlock সৃষ্টি করতে পারে। কিভাবে avoidance অ্যালগরিদম কাজ সম্পর্কে একটি উদাহরণ আছে, আমরা খুব সহজ উদাহরণ নেব।

(স্লাইড সময় : 00:56 দেখুন)

সুতরাং, আসুন আমরা বলি যে আমরা A বিন্দু থেকে যেতে চাই, একটি বিন্দু B তে, একাধিক রুট নিতে পারে আমাদের সেখানে। সুতরাং, এটি একটি নির্দিষ্ট রুট এবং এটি অন্য রুট। এখন আমাদের একটি পছন্দ বা পরিবর্তে সঠিক পছন্দ করতে হবে যা আমাদের এই বিশেষ স্থানে B তে নিরাপদে নিয়ে যাবে। উদাহরণস্বরূপ, যদি আমরা এই রুটের মধ্য দিয়ে যেতে পারি তবে এটি বিপজ্জনক হতে পারে বা রাস্তায় কিছু নির্দিষ্ট বাধা থাকতে পারে। অন্যদিকে তৃতীয় দিকে যাওয়ার ফলে অনুরূপ পরিস্থিতি হতে পারে। সুতরাং, একাধিক উপায় যা আমরা এক

বিন্দু থেকে অন্য দিকে যেতে পারি, আমাদের যে পথে যেতে হবে তা বেছে নিতে হবে। Deadlock avoidance আলগোরিদিম অনুরূপ উপায়ে কাজ করে। আসুন দুটি প্রসেসের সাথে একটি বিশেষ উদাহরণ দেখি।

(স্লাইড সময় : 02:09 দেখুন)

প্রক্রিয়া 1 এবং প্রক্রিয়া 2, এই বিশেষ গ্রাফ সিস্টেমের state প্রতিনিধিত্ব করবে। এখন x অক্ষে প্রক্রিয়া 1 দ্বারা চালানো নির্দেশাবলী এবং y অক্ষে প্রক্রিয়া 2 দ্বারা পরিচালিত নির্দেশাবলির সাথে সম্পর্কিত; উভয় সময়ের সাথে সাথে। এইভাবে, প্রসেস 1 এবং প্রসেস 2 এখান থেকে execution করা শুরু করে যেটি এই বিন্দুটি এখানেই এবং তারা অন্য প্রান্তে execution সম্পূর্ণ করে যা এখানে পয়েন্ট করে। এখন, যেহেতু এই প্রক্রিয়াটি সিস্টেমের মধ্যে execution হয় সেক্ষেত্রে সিস্টেমের state পরিবর্তন হবে এবং এই state পরিবর্তন দুটি প্রক্রিয়া একে অপরের সাপেক্ষে করে কিভাবে চলবে তার উপর নির্ভর করে। সুতরাং আসুন আমরা বলি এই state টি এই ব্লক দ্বারা প্রতিনিধিত্ব করে এখানে। এখন, এখানে এই লাইনটি execution path কে নির্দেশ করে। উদাহরণস্বরূপ, এখানে লাইন horizontal হয় নির্দেশ করে যে নির্দেশাবলী প্রক্রিয়া 1 টি কার্যকর করা হয়। সুতরাং, প্রক্রিয়া 1 execute করে এবং যখন প্রক্রিয়া 2 execute করে না, আমরা এখানে একটি সিস্টেমের অবস্থা পেয়েছি যা এই ছোট বর্গক্ষেত্র দ্বারা দেখানো হয়েছে, তারপর এখানে এই state পরিবর্তিত হয়েছে এবং তারপর এই নির্দিষ্ট সময়ে একটি context switch হয়েছে এবং প্রক্রিয়া 2 execute করে। সুতরাং প্রক্রিয়া 2 কার্যকর হওয়ার ফলে, সিস্টেমের অবস্থা এই সময়ে স্থানান্তরিত হয়ে যায়। তারপর প্রক্রিয়া 1 আবার execute করে এবং state পরিবর্তন হয়, তারপর প্রক্রিয়া 2 চলে, প্রক্রিয়া 1 চলে, ইত্যাদি। এবং শেষে আমাদের উভয় প্রক্রিয়া 1 এবং প্রক্রিয়া 2 এই নির্দিষ্ট পয়েন্ট পৌঁছবে; মূলত উভয় প্রসেসই execute করা হয়েছে। এখন, কিভাবে অপারেটিং সিস্টেমের schedule করে, কিভাবে বিভিন্ন resource allocation বা de-allocation করা হয় প্রক্রিয়া 1 এবং প্রক্রিয়া 2 -এর জন্য, প্রক্রিয়া 2 এর সাপেক্ষে প্রক্রিয়া 1 এর এই নির্দিষ্ট trace বা প্রসেস trace এর উপর নির্ভর করে বিভিন্ন হয়। উদাহরণস্বরূপ, খারাপ ক্ষেত্রে প্রক্রিয়া 1 সম্পন্ন না হওয়া পর্যন্ত ক্রমাগতভাবে চলে, তারপরে প্রক্রিয়া 2 সম্পন্ন না হওয়া পর্যন্ত চলে অথবা অন্য পদ্ধতিটি প্রক্রিয়াটি 1 কিছু সময় চলে তারপর প্রক্রিয়া 2 টি চলে করে, তারপর প্রক্রিয়াটি 1, প্রক্রিয়া 2, প্রসেস 1, ইত্যাদি, যতক্ষণ না উভয়ই সম্পূর্ণ হয়।

(স্লাইড সময় : 05:08 দেখুন)

এখন, আমাদের এটিকে বলা হয় যে প্রক্রিয়া 1 এবং প্রক্রিয়া 2 এর কার্যকারিতা চলাকালীন একটি resource R1 এর প্রয়োজন। সুতরাং, প্রক্রিয়া 1 এই নির্দিষ্ট সময়ে resource R1 প্রয়োজন। সুতরাং, এখানে এই নির্দিষ্ট সময় এই সময়ে দেখানো হয়, তাই প্রক্রিয়া 1 এই resource R1 ব্যবহার প্রয়োজন। সুতরাং, এখানে এই বেগুনি ব্লক দ্বারা highlight করা হয়েছে। এখন, এই সময়ে এই সময়ে এই বিন্দু থেকে একই সময়ে, প্রক্রিয়া 2 resource R1 এর ব্যবহার প্রয়োজন, তাই এটি এখানে highlight করা হয়েছে। এখন এই ছোট বর্গক্ষেত্রটি সেই অঞ্চল যেখানে প্রক্রিয়া

1 এবং পাশাপাশি প্রক্রিয়া 2, এর প্রয়োজন resource R1 কে। তাই, এই দুটি অঙ্কের ছেদচিহ্ন দ্বারা প্রতিনিধিত্ব করে এই ছেদবিশিষ্ট বর্গক্ষেত্র উভয় প্রক্রিয়ার নির্দেশাবলীর অনুরূপ যা resource R1 অনুরোধ করে।

(স্লাইড সময় : 06:15 দেখুন)

একইভাবে, আসুন আমরা বলি যে আরেকটি resource R2 আছে যা প্রক্রিয়া 1 এর এই বিশেষ নির্দেশগুলিতে প্রয়োজনীয় এবং প্রক্রিয়া 2 তে এই বিশেষ নির্দেশগুলিতে প্রয়োজনীয় এবং এটি উল্লিখিত একটি ছেদিত বর্গক্ষেত্র যা এখানে দেখানো হয়েছে এবং এই বর্গক্ষেত্র অংশটি প্রতিনিধিত্ব করে যেখানে উভয় প্রক্রিয়াই p1 এবং p2 resource R2 কে অনুরোধ করা হয়। এখন, আমরা R1 এবং R2 resource গুলির ছেদ অংশ দেখব। সুতরাং, যদি আমরা এই গ্রাফের সাথে এই গ্রাফকে ছেদ করি, তবে আমরা এই পয়েন্টগুলি পাই যেখানে উভয় প্রসেস R1 এর অনুরোধ জানায় যে এখানে এই বেগুনি অংশ, এবং এই অংশটি যেখানে উভয় প্রক্রিয়াগুলির R2 প্রয়োজন, এবং ছেদ এলাকা যেখানে উভয় প্রক্রিয়ার জন্য উভয় resource R1 এবং R2 প্রয়োজন।

(স্লাইড সময় : 06:58 দেখুন)

এখন, এই এলাকাটি একটি deadlock এর কারণ হতে পারে। সুতরাং, আমরা এটি একটি unsafe state বলব। সুতরাং, একটি unsafe state, একটি deadlock state নয়, বরং এটি একটি state যা সম্ভাব্য একটি deadlock এর কারণ হতে পারে। এইভাবে, যদি OS টি প্রক্রিয়া 1 এবং প্রক্রিয়া 2 নির্ধারিত হয় তবে এই ধরনের unsafe state অবস্থায় execution পথটি শেষ হয়ে যায়, তাহলে আমরা সম্ভবত একটি deadlock পেতে পারি। অন্যদিকে, যদি OS প্রক্রিয়া 1 এবং 2 এভাবে চালায় যে এই unsafe state এড়িয়ে যাওয়া হয় তবে deadlock এড়িয়ে যাওয়া যাবে। সুতরাং, এটি সিস্টেমের মধ্যে deadlock এড়াতে ব্যবহৃত প্রয়োজনীয় কৌশল।

(স্লাইড সময় : 08:33 দেখুন)

তাই পরবর্তী প্রশ্নটি হল- একটি অ্যালগরিদম আছে কি যা চিরস্থায়ীভাবে সঠিক পছন্দ করে deadlock এড়াতে পারে? যে আমরা প্রসেস execute বা resource allocation এর কিছু উপায় পছন্দ করে একটি সিস্টেম unsafe state কখনো না পৌঁছানোর নিশ্চিত করতে পারি। সুতরাং, আমরা এমন একটি অ্যালগরিদম নির্মাণ করতে পারি যা Banker's Algorithm নামে পরিচিত। সুতরাং, একটি উদাহরণ দেখা যাক।

(স্লাইড সময় : 09:12 দেখুন)

এখন, 3 টি client A, B এবং C সহ একজন banker বিবেচনা করুন। প্রতিটি ক্লায়েন্টের নির্দিষ্ট credit limit আছে এবং এই সমস্ত credit limit গুলির যোগফল 20। এখন ব্যাংকার জানে যে সর্বাধিক 20 টি ক্রেডিট একযোগে ব্যবহার করা হবে না তাই তিনি শুধুমাত্র 10 ইউনিট রাখেন। এর অর্থ হল, আসুন আমরা 3 জন ক্লায়েন্ট

A, B এবং C এর সাথে এই টেবিলে বিবেচনা করি এবং প্রতিটি ক্লায়েন্টের অ্যাকাউন্টে কিছু ইউনিট আছে। সুতরাং, উদাহরণস্বরূপ, A- এর 3 টি ইউনিট আছে, B 2 টি ইউনিট পেয়েছে এবং C 2 টি ইউনিট পেয়েছে, তাই মোট 4 5 6 7। এবং যেহেতু, ব্যাংকার শুধুমাত্র 10 টি ইউনিট রাখে তাই অবশিষ্ট 3 টি ইউনিট উপলব্ধ আছে এবং কোনও ক্লায়েন্টকে allocate করা হয়নি। অন্যদিকে, প্রতিটি ক্লায়েন্টের জন্য সর্বাধিক ক্রেডিট যথাক্রমে 9, 4 এবং 7। সুতরাং, আপনি মনে রাখবেন 9 প্লাস 4 প্লাস 7 সর্বাধিক ক্রেডিট সীমা যা 20 ইউনিট এর সমষ্টি। এবং এই সর্বোচ্চ ক্রেডিট সীমা অগ্রিম ক্লায়েন্ট দ্বারা ঘোষণা করা হয়। এখন, এই নির্দিষ্ট টেবিলের দেওয়া ব্যাংকারকে ব্যবহারকারীদের সর্বোচ্চ ক্রেডিট জমা দিতে হবে যাতে কোন unsafe state না হয়। সুতরাং, আসুন আমরা উদাহরণগুলি দেখি।

(স্লাইড সময় : 11:07 দেখুন)

তাই, আমরা যে সমস্যাটি এখানে সমাধান করার চেষ্টা করছি তা হচ্ছে এই A, B, C এর সর্বোচ্চ allocation গুলি 9, 4 এবং 7 এভাবে করা যায় যাতে state safe হয়। সুতরাং, আমরা 2 ইউনিটকে B কে allocation দিয়ে শুরু করি। তাই, আমাদের 3 টি ইউনিট উপলব্ধ আছে, এই 3 টি ইউনিট থেকে 2 টি B কে allocate করা হয়। সুতরাং B এর 2 যোগ 2, 4 টি ইউনিট রয়েছে এবং উপলব্ধ এখন 3 বিয়োগ 2 যা 1 ইউনিট। এখন, B শেষ হওয়ার পর সমস্ত 4 টি ইউনিট free store এ ফেরত পাঠায়, তাই 4 টি যোগ 1 টি free store এ 5 টি ইউনিট থাকবে এবং এগুলি 5 টি ইউনিট C-এ allocate করা যাবে। সুতরাং, C এর সর্বোচ্চ ক্রেডিট সীমা হবে 5 প্লাস 2 যে এখানে 7টি ইউনিট দেখানো। সুতরাং, free store পরিমাণটি 0 ইউনিট। এখন, C এর সমস্ত 7 টি ইউনিট সম্পন্ন হলে C free store এ ফিরে আসে। এখন ব্যাংকার A দ্বারা প্রয়োজনীয় resource allocate করতে পারে এবং A এর সর্বাধিক resource প্রয়োজন হয়, যেটি 7 টি ইউনিটে পেয়ে সর্বোচ্চ ক্রেডিট সীমা 9 ইউনিট পেতে পারে। সুতরাং, আমাদের কাছে 9 টি A কে allocate করা এবং 1 টি ইউনিট উপলব্ধ। এইভাবে আমরা এই বিশেষ শুরু অবস্থা বা সিস্টেমের এই বিশেষ অবস্থা, যেখানে 3 টি ইউনিট উপলব্ধ আছে এবং সর্বোচ্চ 9, 4 এবং 7 ইউনিট আছে যা প্রতিটি ক্লায়েন্টের ক্রেডিট সীমা থাকতে পারে। আমরা দেখি যে এই বিশেষ state এর সঙ্গে একটি schedule আছে যা সম্ভব হয় যাতে সমস্ত অনুরোধ এবং ক্লায়েন্টদের দ্বারা সর্বাধিক অনুরোধ পূরণ করা যায়। সুতরাং, আমরা এটিকে safe state বলি এবং ব্যাংকার এই ধরনের allocation করতে পারে।

(স্লাইড সময় : 13:35 দেখুন)

এখন, আরেকটি উদাহরণ দেখি যেখানে আমরা যে অবস্থা শুরু করি তা হল A এর 4 টি ইউনিট আছে, B এর 2টি এবং C এর 2টি ইউনিট আছে, তাই মোট 8 টি ইউনিট রয়েছে A, B এবং C এর কাছে। এবং ফ্রি ইউনিট সংখ্যা 2 এবং প্রত্যেক ক্লায়েন্টের সর্বোচ্চ ক্রেডিট সীমা হল যথাক্রমে 9, 4 এবং 7। এখন, আমরা দেখতে পাচ্ছি যে ব্যাংকার কিভাবে প্রতিটি গ্রাহকের জন্য এই অবশিষ্ট ইউনিট allocate করার চেষ্টা করছেন, এমন কোন schedule

নেই যা এই প্রয়োজনীয়তাগুলি পূরণ করতে পারে। উদাহরণস্বরূপ ধরি B কে 2 ইউনিট আরও allocate করা হয় এবং শুধুমাত্র 2 টি ইউনিট free আছে দেখুন, তাই B একমাত্র ক্লায়েন্ট যাকে দেওয়া যাতে পারে। অতএব, B 4 ইউনিট পায় যখন সেখানে 0 টি free ইউনিট রয়েছে। এখন B শেষ হওয়ার পর আমাদের 4 টি ইউনিট free আছে, কিন্তু আপনি দেখতে পারেন A যার 5 ইউনিট দরকার বা C যার 5 ইউনিট দরকার আর 4 টি ইউনিট উপলব্ধ আছে যা তাদেরকে দেওয়া যায় না। অতএব, আমরা দেখতে পাচ্ছি যে এই state টি একটি unsafe state কারণ সেখানে কোনও schedule নেই যা এই প্রয়োজনীয়তাগুলি পূরণ করতে পারে। সুতরাং, এটি একটি unsafe state এবং ব্যাংকারকে অবশ্যই নিশ্চিত করতে হবে যে এই ধরনের কোনো state পৌঁছেছে না।

(স্লাইড সময় : 15:33 দেখুন)

তাই একই পদ্ধতিটি অপারেটিং সিস্টেমের মধ্যে প্রয়োগ করা যায়, যেখানে প্রতিটি request এর সাথে সংশ্লিষ্ট অপারেটিং সিস্টেম নির্ধারণ করবে যে সিস্টেমটি একটি safe state এ থাকে কিনা। এবং যদি প্রকৃতপক্ষে সিস্টেমটি safe state এ অবস্থায় থাকে তবে এটির একটি নির্দিষ্ট schedule থাকে যাতে সমস্ত request grant করা যায়, তারপর request গুলি allocate করা হয়। অন্যথায় request টি স্থগিত করা হয়।

(স্লাইড সময় : 16:08 দেখুন)

Deadlocks প্রতিরোধের তৃতীয় উপায় কে বলা হয় Deadlock Prevention (প্রতিরোধ)। মূলত আমরা সিস্টেম design দ্বারা deadlock প্রতিরোধ করি যেখানে 4 টি শর্তের একটি পূরণ হয় না। সুতরাং, আমরা এই 4 টি শর্ত দেখেছি এবং যা একটি deadlock ঘটতে অপরিহার্য। সিস্টেমের মধ্যে এই শর্তগুলির একটি প্রতিরোধ করে, deadlock প্রতিরোধ করতে পারি। উদাহরণস্বরূপ, যদি আমরা এমন একটি সিস্টেম ডিজাইন করি যেখানে hold and wait পূরণ না হতে পারে, অর্থাৎ এটি প্রসেস resource এর জন্য অপেক্ষা করার সময় কোনও resource ধরে রাখতে পারে না, এই ধরনের সিস্টেমে কোনও deadlock থাকবে না। সুতরাং, আসুন আমরা এমন বিভিন্ন উপায়ে দেখতে পারি যা আমরা আসলে এইসব শর্তগুলির মধ্যে একটি প্রতিরোধ করতে পারি।

(স্লাইড সময় : 17:10 দেখুন)

সুতরাং mutual exclusion রোধ করা, তাই প্রথাগতভাবে এটি সম্ভব নয়। আমরা সর্বদা mutual exclusion বর্জন করতে সক্ষম হব না। উদাহরণস্বরূপ, একটি printer resource ক্ষেত্রে এটি একটি বিশেষ প্রক্রিয়ার সাথে সংশ্লিষ্ট printing করে। সুতরাং, এটি একযোগে দুই প্রসেসের সাথে share করা যাবে না যদি একটি spool উপস্থিত না থাকে। যাইহোক, এই অবস্থায়, অপারেটিং সিস্টেম optimal resource allocation নিশ্চিত করতে পারে। সুতরাং Hold and Wait দেখা যাক। সুতরাং, এটি অর্জন করার একটি উপায় হল সমস্ত প্রসেসগুলি execution এর শুরুতে তাদের resource request করবে। সুতরাং, এটা স্পষ্টতই সম্পদগুলির optimal ব্যবহারের জন্য নয়

এবং execution হওয়ার সময় resource requirement জানা সম্ভব নাও হতে পারে। এখন, তৃতীয় উপায় হল No Preemption এর দিকে তাকাই। সুতরাং, printer spool দ্বারা resource virtualization দ্বারা যেমন resource preempt । সুতরাং, একটি printer spool এর মধ্যে আমরা একই সময়ে বিভিন্ন প্রক্রিয়ার ডকুমেন্টগুলি মুদ্রণ করতে পারি। সুতরাং, এই সমস্ত document গুলি printer এ spool বা buffer করা হয় এবং একে একে print করা হয়। আমরা লক্ষ্য করতে পারি চূড়ান্ত কৌশল Circular Wait প্রতিরোধ করা । সুতরাং এটি প্রতিরোধ করার এক উপায় হল যে একটি প্রসেস resource hold করে একই সময়ে অন্য এক resource অনুরোধ করতে পারেন না। Circular wait প্রতিরোধের দ্বিতীয় উপায় একটি নির্দিষ্ট ক্রমে অনুরোধ resource request হয় – sequential বা hierarchical ।

(স্লাইড সময় : 19:20 দেখুন)

তাই আসুন আমরা hierarchical ক্রমটি দেখি। এখানে resource গুলি এমন স্তরে অন্তর্ভুক্ত করা হয় যা তাদের কিছু numeric value দ্বারা priority দেওয়া হয়। একটি প্রক্রিয়া শুধুমাত্র বর্তমানে যে resource hold করছে তার থেকে উচ্চতর স্তরের resource গুলির জন্য অনুরোধ করতে পারে এবং resource গুলি কোনও ক্রমে release করা যেতে পারে। উদাহরণস্বরূপ, আসুন আমরা বলি যে priority এর ঊর্ধ্বক্রমে semaphores s1, s2 এবং s3 আছে, s3 সর্বোচ্চ priority, যেখানে s1 হল সর্বনিম্ন priority । সুতরাং, আসুন আমরা বলি একটি প্রক্রিয়া semaphores এর এই বিশেষ ক্রমে s1,s2 এবং s3 এর request সূত্রপাত করে। সুতরাং, আমরা দেখি যে একটি ক্রম অনুসরণ করা হয় s2 s1 এর চেয়ে বড় এবং s3 s2 এর চেয়ে বড়। সুতরাং, এই ধরনের allocation দেওয়া উচিত। এই ক্ষেত্রে যেখানে আমাদের প্রথম s1 আছে তারপর s3 এবং তারপর s2, তাই এই বিশেষ ক্ষেত্রে এই allocation অনুমোদিত নয় কারণ ক্রম বজায় থাকে না। ধন্যবাদ।